IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

PROJET I.O.T

Capteur intelligent TempHumiX

Simulation, communication WiFi via MQTT
et IHM d'affichage et d'archivage des données

Compte rendu de travaux pratiques

Javier Santiago VARGAS PARRA

Poste n° 44987

Encadrant : Abdellah BOUCHAMA

IMT Nord Europe – CERI Systèmes Numériques

Novembre 2026

Sommaire

1 Introduction 2
2 TP 1 – Conception du capteur en simulation 3
2.1 Démarche par briques 3
2.2 Le capteur TempHumiX simulé 3
3 TP 2 – Communication WiFi via MQTT 4
3.1 Prise en main du protocole avec MQTTX 4
3.2 Portage sur la carte réelle Arduino UNO R4 WiFi 4
3.3 Publication et souscription MQTT 5
3.4 Amélioration de la publication 6
3.5 Analyse des trames avec Wireshark 6
3.6 Questions générales 8
4 TP 3 – L'IHM d'affichage des données 8
4.1 Environnement et architecture logicielle 8
4.2 Clients MQTT en mode console 8
4.3 Réception dans un thread 10
4.4 Conception de l'interface 10
4.5 Programme principal 11
4.6 Notice de fonctionnement de l'interface 12
4.7 Archivage dans SQLite 13
4.8 Affichage des courbes 14
5 Difficultés rencontrées 15
6 Conclusion et perspectives 15

1 Introduction

Le but du projet est de fabriquer un capteur de température et d'humidité capable de communiquer, le boîtier TempHumiX, puis d'écrire le logiciel qui permet de le suivre depuis un PC. Nous l'avons fait en trois TP, chacun repartant du résultat du précédent.

Le premier TP se passe entièrement en simulation, sous SimulIDE, avec un Arduino UNO R3. Dans le deuxième, on porte le programme sur le vrai boîtier (Arduino UNO R4 WiFi), on le connecte au réseau et on le fait parler en MQTT. Le troisième TP, celui auquel nous avons consacré le plus de temps et qui occupe la plus grande partie de ce rapport, consiste à écrire en Python une interface qui affiche les mesures, envoie des commandes au capteur, archive les données et trace des courbes.

Le capteur et le PC ne communiquent jamais directement. Tout passe par le broker Mosquitto du réseau IoT de la salle : chacun publie ou s'abonne à des topics qui se terminent par le numéro de notre poste, `44987`. Le code est suivi sous Git, avec un commit par étape. Les mots de passe WiFi et MQTT sont dans des fichiers à part, exclus du dépôt (`arduino_secrets.h` et `secrets_mqtt.py`).

***Tableau 1** – Infrastructure réelle du réseau IoT*

Élément	Valeur	Remarque
Réseau WiFi	IoT IMT Nord Europe	2,4 GHz seulement : l'ESP32-S3 de la UNO R4 n'a pas de radio 5 GHz

Élément	Valeur	Remarque
Broker MQTT	`192.168.1.1:1883`	Mosquitto 2.1.2, sans TLS, avec utilisateur et mot de passe
Serveur NTP	`192.168.1.254`	Celui de l'énoncé (10.12.130.1) ne répond pas depuis le réseau IoT
Capteur	`192.168.1.104` (DHCP)	Heure locale UTC+1 (heure d'hiver)

2 TP 1 – Conception du capteur en simulation

2.1 Démarche par briques

Plutôt que de dessiner tout le circuit d'un coup, nous avons d'abord testé chaque fonction dans un petit projet séparé. Chaque projet a son sous-répertoire, avec un fichier `.sim1` et un fichier `.ino` du même nom, sinon SimulIDE ne retrouve pas le programme. Une fois les briques validées, il ne restait plus qu'à les assembler dans TempHumiX.

Tableau 2 – *Mini-projets réalisés sous SimulIDE*

Projet	Ce qu'il fait	Résultat
`clignotement_led`	Une LED sur la broche 2 qui change d'état toutes les 500 ms	Premier circuit, pour prendre en main la suite compiler, charger, simuler
`bargraphe`	Chenillard sur les LED des broches 5 à 12, une seule allumée à la fois	Le chenillard tourne, mais avec un pas de 500 ms dans notre version
`capteur_dht22`	Lecture du DHT22 toutes les 2 s avec la bibliothèque DHT d'Adafruit	Une LED s'allume sous 20 °C, l'autre au-dessus
`lcd_i2c`	Afficheur 16×2 AIP31068 en I2C, adresse 0x3E	« Hello » / « World ». L'en-tête à inclure est `LiquidCrystal_AIP31068_I2C.h`
`lcd_i2c_dht22`	LCD et DHT22 ensemble	« IMT NE - PPI » puis « T 22.5oC/H 68.5% »

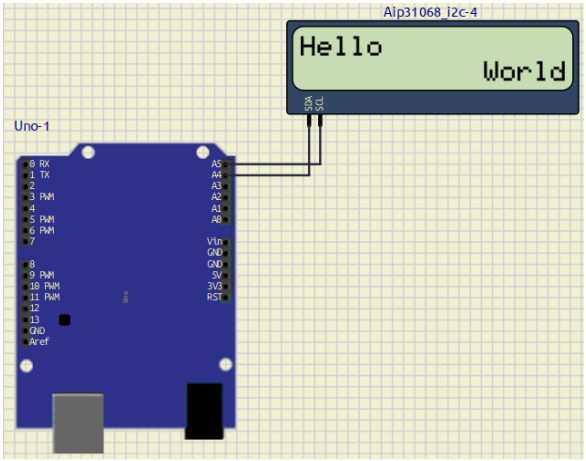


Figure 1 – *Circuit lcd_i2c en simulation*

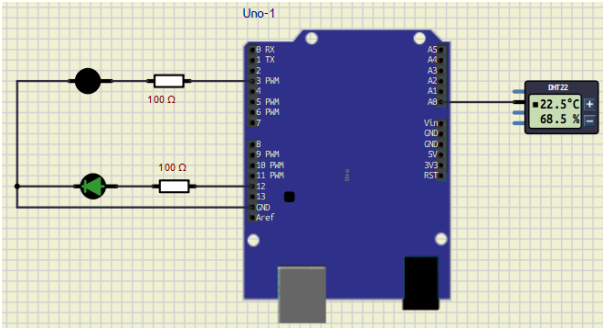


Figure 2 – *Circuit capteur_dht22 : 22,5 °C > 20 °C*

2.2 Le capteur TempHumiX simulé

Le circuit final réunit l'Arduino UNO R3, le DHT22, le LCD I2C, le bargraphe de 10 LED et son réseau de résistances, la LED de vie et un port série. Pour éviter les `delay()` qui bloquent tout, le programme tourne sur une base de temps de 100 ms : un compteur de ticks décide de ce qui doit être fait à chaque passage.

```
digitalWrite(PIN_LED_VIE, (tick % 5) < 2 ? HIGH : LOW);  // LED de vie : période 500 ms = 2 Hz
if (tick % 10 == 0) avancerDate();                        // dateur : +1 s tous les 10 ticks
if (tick % 20 == 0) { /* lecture DHT22, bargraphe, LCD ligne 2, port série */ }
tick++;  delay(100);
```

Le dateur est fait à la main : secondes, minutes, heures, jours, mois et années, avec des mois de 30 jours comme le permet l'énoncé. Chaque seconde, la première ligne du LCD affiche la date. Toutes les deux secondes, la seconde ligne affiche la température et l'humidité au dixième, les mêmes valeurs partent sur le port série, et le bargraphe allume une LED par tranche de 10 % d'humidité.

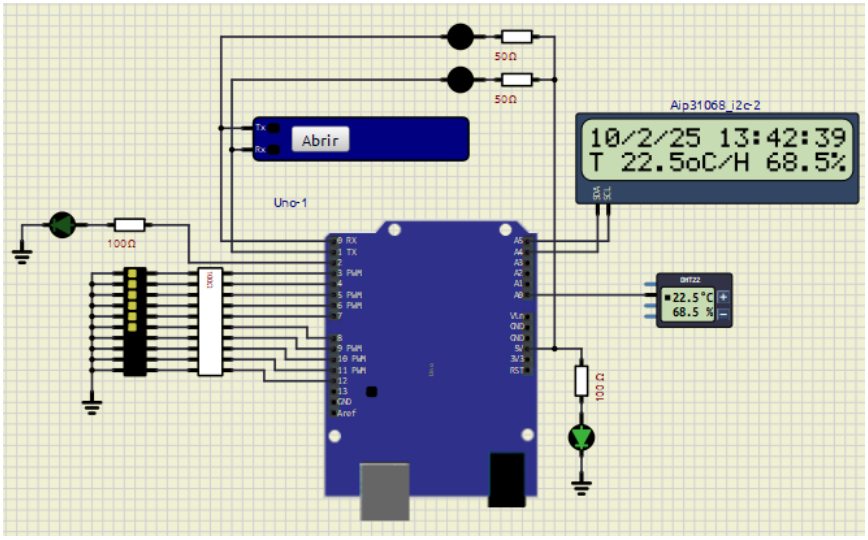


Figure 3 – *TempHumiX en simulation : « 10/2/25 13:42:39 » et « T 22.5oC/H 68.5% », 6 LED du bargraphe pour 68,5 %*

Que retenir de ces circuits ? Le découpage nous a fait gagner du temps. Quand le montage final ne marchait pas, on savait déjà que chaque élément fonctionnait seul, et il ne restait qu'à chercher du côté de l'assemblage ou du découpage du temps. La simulation a quand même ses limites : la valeur du DHT22 se règle à la main dans le composant et les temps d'exécution ne sont pas ceux d'une vraie carte. On s'en est rendu compte plus tard, sur le matériel.

3 TP 2 – Communication WiFi via MQTT

3.1 Prise en main du protocole avec MQTTX

Avant de toucher au capteur, nous avons manipulé le broker à la main avec MQTTX : connexion « IOT Network » avec `mos_user` et un Keep Alive de 60 s, puis publication et abonnement sur `TESTS/TEST_44987`. Les essais du §5.2.4 ont été instructifs.

- En s'abonnant à la variable d'un autre poste, on reçoit ses messages sans aucune restriction. Le broker relaie à tous ceux qui sont abonnés, les clients ne se connaissent pas, et la règle d'accès est la même pour tout le monde.
- Avec `TESTS/#`, le joker `#` couvre tous les niveaux en dessous : on voit passer les messages de toute la salle.
- Avec l'option Retain, le broker garde le dernier message du topic et le donne tout de suite à quiconque s'abonne ensuite, même longtemps après la publication.

3.2 Portage sur la carte réelle Arduino UNO R4 WiFi

Passer de la UNO R3 à la UNO R4 a demandé trois changements. L'écran du boîtier utilise la bibliothèque Grove LCD RGB Backlight (`rgb_lcd`). Le dateur logiciel a disparu au profit du RTC du microcontrôleur Renesas RA4M1, qui compte tout seul et connaît le vrai calendrier. Enfin, la boucle est devenue une machine d'état à 500 ms, ce qui fait clignoter la LED de vie à 1 Hz et non plus à 2 Hz, puisqu'elle change d'état à chaque pas. Petit détail d'affichage : `21/10/26 14:00:05` fait 17 caractères pour 16 colonnes, donc on supprime l'espace quand le jour et le mois ont deux chiffres.

Pour le WiFi, `setup_wifi()` (bibliothèque `WiFiS3`) essaie trois fois de se connecter et renvoie `true` ou `false`. Sur « IoT IMT Nord Europe 5G », ça ne marche pas : le module ESP32-S3 n'a qu'une radio 2,4 GHz et le réseau 5 GHz n'apparaît même pas dans le balayage. La mise à l'heure passe par `NTPClient`. Nous avons mis du temps à comprendre pourquoi elle échouait, jusqu'à découvrir que le serveur de l'énoncé n'était pas joignable depuis ce réseau ; le serveur local `192.168.1.254` fonctionne. L'appel NTP attend au plus une seconde. S'il échoue, le LCD affiche « Pas synchro NTP », la mesure continue et on réessaie une minute plus tard.

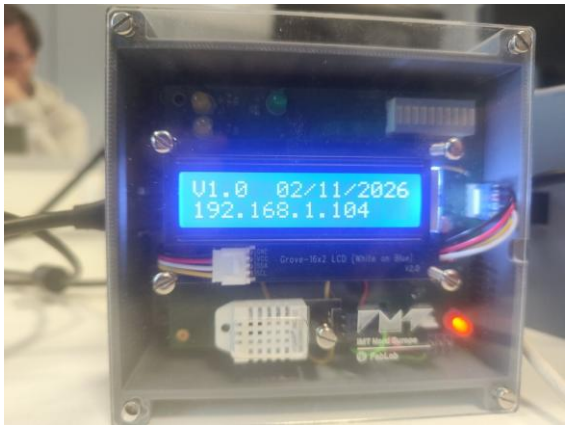


Figure 4 – *Écran d'accueil : version, date et IP DHCP*

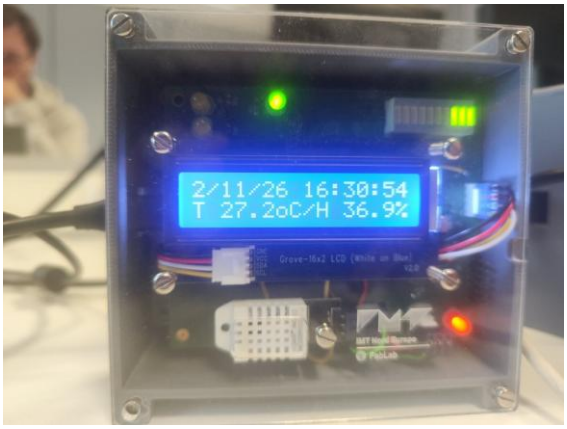


Figure 5 – *Date du RTC synchronisé et T/H*

3.3 Publication et souscription MQTT

`setup_mqtt()` connecte le client `ArduinoMqttClient` sous l'identifiant `IDO_Client_44987` et renvoie 0, ou le code d'erreur MQTT en cas d'échec. Le capteur publie quatre topics et en écoute trois. À chaque tour de boucle, `mqttClient.poll()` lit les messages arrivés et entretient la connexion.

Tableau 3 – *Topics et commandes (préfixe /iot/, suffixe _44987)*

Topic	Sens	Contenu
`VERSION_`	publié	Version « X.Y » (1.0)
`TEMPERATURE_` / `HUMIDITY_`	publié	Valeurs au dixième (ex. 23.4 et 37.0)
`GET_DATE_`	publié	« JJ/MM/AAAA HH:MM:SS » lue dans le RTC
`COMMAND_`	souscrit	10 mise à l'heure NTP · 20/30 démarrage/arrêt de la mesure · 40/50 clignotement lent/rapide · 60 clignotement personnalisé
`BARGRAPH_`	souscrit	0 à 1023, ramené à 0 à 10 LED avec `map()`
`BLINK_`	souscrit	Période en ms, refusée si ce n'est pas un multiple de 100 ms

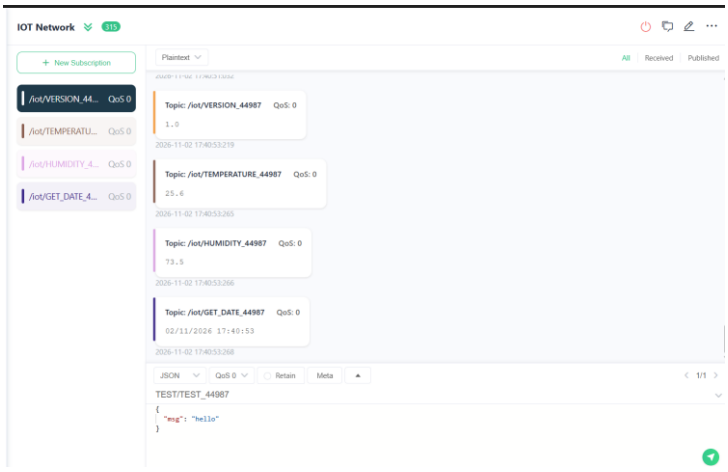


Figure 6 – *Les quatre topics publiés reçus dans MQTTX (02/11, 17:40:53)*

Quand un message arrive, `onMqttMessage()` récupère le topic, convertit le texte en entier et l'envoie au bon traitement. Pour accepter des périodes réglables par pas de 100 ms, nous sommes repassés à une base de temps de 100 ms (`TICK_MS`), la machine d'état n'avançant qu'un tick sur cinq.

```
int valeur = atoi(message);
if (topic.indexOf("COMMAND") >= 0)       traiterCommande(valeur);
else if (topic.indexOf("BARGRAPH") >= 0) { valeurBargraphe = constrain(valeur, 0, 1023);
afficherBargraphe(); }
else if (topic.indexOf("BLINK") >= 0) {
  if (valeur >= TICK_MS && valeur % TICK_MS == 0) periodePerso = valeur;
  else Serial.println("Periode refusee : multiple de 100 ms attendu");
}
```



Figure 7 – *BARGRAPH = 512 : 5 LED sur 10*

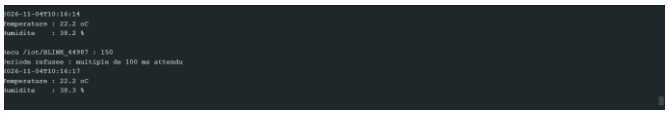


Figure 8 – *BLINK = 150 refusé (moniteur série)*

Une erreur bête nous a coûté un moment : le premier essai depuis MQTTX était envoyé en JSON (`{"msg": 512}`). `atoi()` ne sait pas lire ça, il renvoyait 0 et le bargraphe s'éteignait. Les messages du projet sont du texte brut, il suffit d'envoyer le nombre seul.

3.4 Amélioration de la publication

Publier les quatre topics toutes les deux secondes chargeait le réseau pour rien. Désormais la version n'est envoyée qu'après une mise à l'heure réussie (commande 10), et la température, l'humidité et la date ne partent que si T ou H a changé au dixième près. Pour la comparaison, nous passons en dixièmes entiers avec `lround(x * 10)`, ce qui évite de comparer des flottants. Les commandes 30 et 20 arrêtent puis relancent la mesure ; pendant l'arrêt, le LCD affiche « Mesure arretee ».

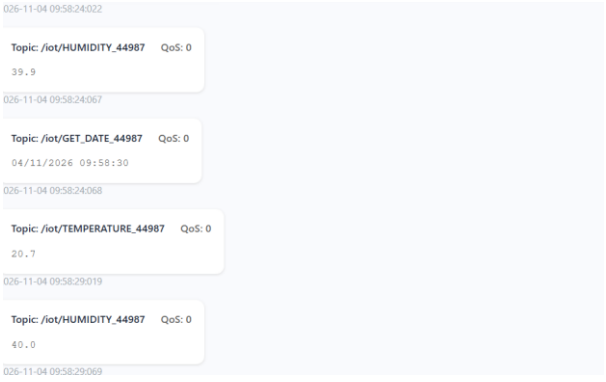


Figure 9 – *Publication seulement quand H passe à 40,0 %*

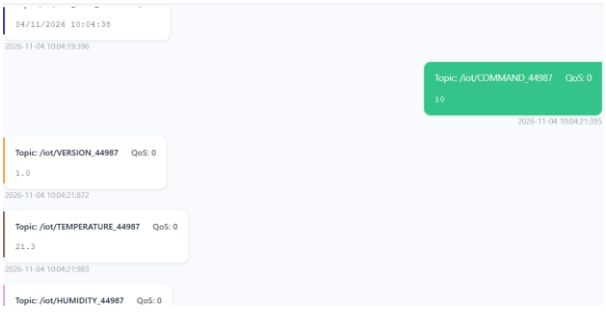


Figure 10 – *Commande 10 : version reçue 0,48 s après*

3.5 Analyse des trames avec Wireshark

Nous avons capturé une session complète entre MQTTX (PC `192.168.1.168`) et le broker, avec le filtre de capture `tcp port 1883` puis le filtre d'affichage `mqtt`.

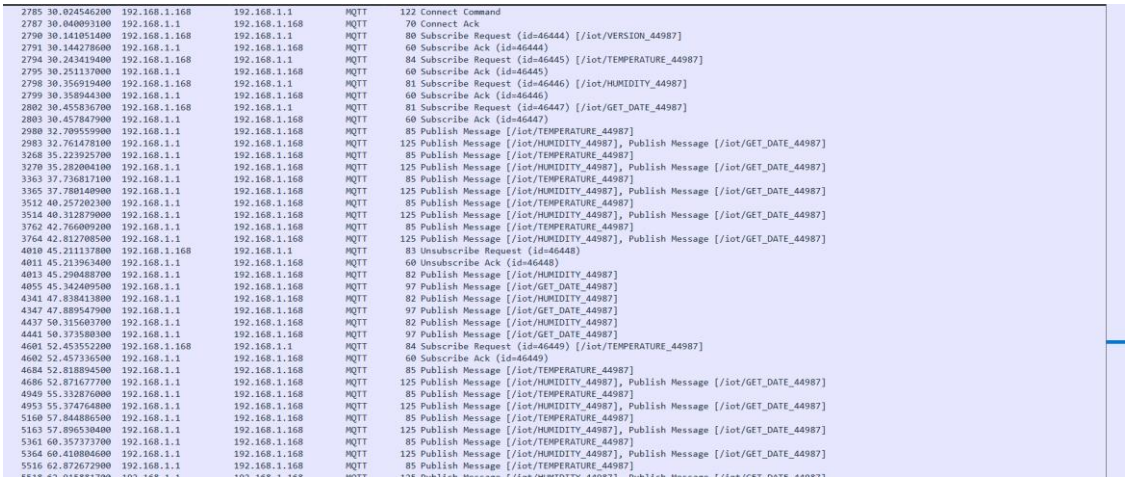


Figure 11 – *Début de session : CONNECT, CONNACK, 4 SUBSCRIBE/SUBACK puis PUBLISH*

Tableau 4 – *Relevé des paquets MQTT*

Paquet	Sens	Champs relevés
CONNECT	client → broker	MQTT v5.0, Keep Alive 60 s, Client ID `mqttx_c5313646_…`, User Name et Password `mos_user` en clair
CONNACK	broker → client	Reason Code 0 (Success), 16 ms après le CONNECT
SUBSCRIBE / SUBACK	client ↔ broker	Un identifiant par topic (46444 à 46447), QoS demandé et accordé : 0
PUBLISH	broker → client	`/iot/TEMPERATURE_44987`, message « 22.6 », QoS 0, Retain 0, DUP 0
PINGREQ / PINGRESP	client ↔ broker	2 octets MQTT, envoyés quand le client est resté silencieux pendant le Keep Alive
DISCONNECT	client → broker	Reason Code 0 : fin volontaire, pas de Last Will publié

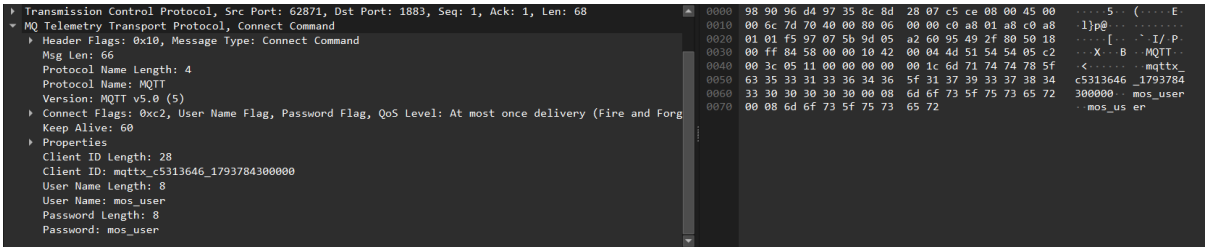


Figure 12 – *Paquet CONNECT : utilisateur et mot de passe lisibles en clair*

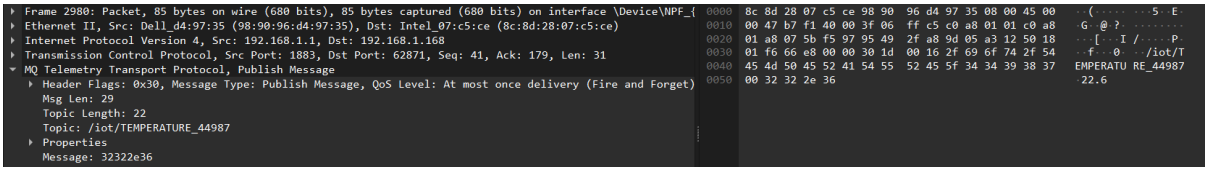


Figure 13 – *Paquet PUBLISH de la température : 85 octets*

Le mot de passe `mos_user` se lit en clair dans le paquet CONNECT, dans le champ Password comme dans la vue hexadécimale. Autrement dit, n'importe qui sur le WiFi peut récupérer les identifiants et toutes les données. Pour sécuriser, il faudrait passer en MQTT sur TLS (port 8883) et donner à chaque objet ses propres identifiants.

Un PUBLISH de température fait 85 octets sur le réseau : 14 pour Ethernet, 20 pour IPv4, 20 pour TCP et 31 pour MQTT (en-tête fixe 2, longueur du topic 2, topic 22, propriétés 1, message 4). La donnée utile, « 22.6 », tient en 4 octets. Une requête HTTP équivalente porterait plusieurs centaines d'octets d'en-têtes et ouvrirait souvent une nouvelle connexion à chaque envoi, alors que MQTT garde la même connexion ouverte.

Le Keep Alive est la durée maximale pendant laquelle le client peut ne rien envoyer. S'il n'a pas de données, il envoie un PINGREQ et le broker répond PINGRESP. Si le broker n'entend rien pendant une fois et demie cette durée, il considère le client comme parti, ferme la session et publie son message de testament s'il en a un.

3.6 Questions générales

Côté modèle OSI, on trouve en application MQTT (port 1883), NTP (123) et DHCP ; en transport, TCP pour MQTT, qui a besoin d'une connexion fiable, et UDP pour NTP et DHCP ; en réseau, IPv4 sur le sous-réseau 192.168.1.0/24 ; en dessous, du WiFi 802.11n à 2,4 GHz entre le capteur et le point d'accès du RUTX50. Concrètement, un PUBLISH du capteur est mis dans un segment TCP, puis dans un paquet IP, envoyé par WiFi au RUTX50, et remis à Mosquitto qui le recopie vers chaque abonné.

[Questions 2 à 5 sur le RUTX50 – à compléter et vérifier dans la documentation Teltonika : services applicatifs (broker MQTT, NTP, DHCP, DNS, VPN, pare-feu…), place dans l'architecture IoT (Fog/Edge, rôle de passerelle), processeur (MPU ARM) et système RutOS basé sur OpenWrt, nombre maximal de clients WiFi, topologie en étoile, mesures de sécurité du dernier firmware.]

Les journaux de Mosquitto 2.1.2 montrent un broker qui démarre avec authentification (`basic-auth`) et contrôle d'accès par topic (`acl-check`). Il se plaint que les fichiers `passwd` et `acl` soient lisibles par tous et n'appartiennent pas à l'utilisateur mosquitto (il propose lui-même `chmod 0700` et `chown`). Le motif d'ACL `TESTS/#` ne contient ni `%c` ni `%u`, donc tous les utilisateurs ont exactement les mêmes droits. Enfin, il écoute en IPv4 et en IPv6 sur le port 1883, c'est-à-dire sans chiffrement, ce qui rejoint ce que nous avons vu dans Wireshark.

4 TP 3 – L'IHM d'affichage des données

4.1 Environnement et architecture logicielle

Nous avons développé sous PyCharm, dans un environnement virtuel en Python 3.12, avec `paho-mqtt` 2.1, `PySide6` (Qt 6), `sqlite3` et `matplotlib`. L'application s'est construite par couches : d'abord un client console qui lit, puis un client console qui écrit, puis les deux réunis grâce à un thread, et enfin la fenêtre, l'archivage et les courbes. Chaque étape réutilise le code de la précédente.

***Tableau 5** – Classes de l'application*

Classe	Fichier	Rôle
`ClientMQTT`	`client_mqtt.py`	Enveloppe paho-mqtt (connexion, publication, souscription, boucle réseau), garde les dernières valeurs reçues et les archive dans SQLite
`ThreadMQTT`	`thread_mqtt.py`	Thread de réception : ouvre la base, s'abonne aux 4 topics et appelle `wait_mqtt(1 s)` en boucle jusqu'à `stopthread()`
`MainWindow`	`main.py` + `mon_ihm.ui`	Fenêtre principale : paramètres du broker, affichage rafraîchi chaque seconde par un `QTimer`, boutons de commande
`CourbeDialog`	`dialogue_courbe.py` + `graphe.ui`	Boîte de dialogue des courbes : requête SQL sur un intervalle de temps et tracé Matplotlib

La version 2 de paho-mqtt nous a obligés à adapter le code de l'énoncé. Le client se crée avec `CallbackAPIVersion.VERSION2`, et dans `on_connect` le code de retour est un objet `ReasonCode` qu'on teste avec `rc.is_failure`. Autre piège : les noms de topics sont sensibles à la casse. L'énoncé les écrit en minuscules (`/iot/temperature_…`) alors que notre capteur publie en majuscules (`/iot/TEMPERATURE_44987`). Avec les noms de l'énoncé, nous n'aurions rien reçu ; le code Python reprend donc exactement ceux du programme Arduino.

4.2 Clients MQTT en mode console

La classe `ClientMQTT` offre cinq méthodes : `connect_mqtt()`, `disconnect_mqtt()`, `publish_mqtt()`, `subscribe_mqtt()` et `wait_mqtt()`. Le client de lecture s'abonne aux quatre topics et affiche chaque message

reçu. Appelée sans délai, `wait_mqtt()` lance `loop_forever()`, qui fait tourner la boucle réseau ; sans elle, aucun rappel ne s'exécute.

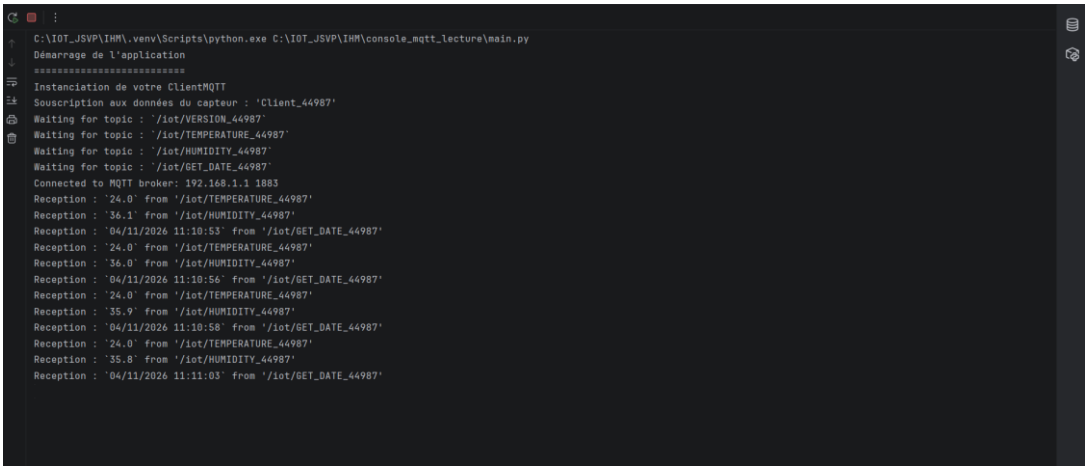


Figure 14 – *Client de lecture : quatre abonnements, connexion, puis réception de T, H et de la date*

Le client d'écriture affiche un menu, et chaque choix publie une ou deux commandes. Il se connecte sous `Client_E44987` : le broker refuse deux clients avec le même identifiant, la deuxième connexion éjecterait la première. Grâce à ce « E », les deux programmes peuvent tourner en même temps.

Tableau 6 – *Menu du client d'écriture*

Choix	Topic	Message(s)
1 – Mise à l'heure	`COMMAND_`	10
2 / 3 – Démarrage / arrêt des mesures	`COMMAND_`	20 puis 50 / 30 puis 40. Comme le demande l'énoncé, la LED de vie montre l'état : rapide si la mesure tourne
4 / 5 – Clignotement lent / rapide	`COMMAND_`	40 / 50
6 – Clignotement personnalisé	`BLINK_` puis `COMMAND_`	Période saisie, puis 60. La valeur doit arriver avant l'ordre
7 – Bargraphe	`BARGRAPH_`	Valeur 0 à 1023, vérifiée avant l'envoi
8 – Afficher les mesures	—	Valeurs gardées par le thread (§4.3)

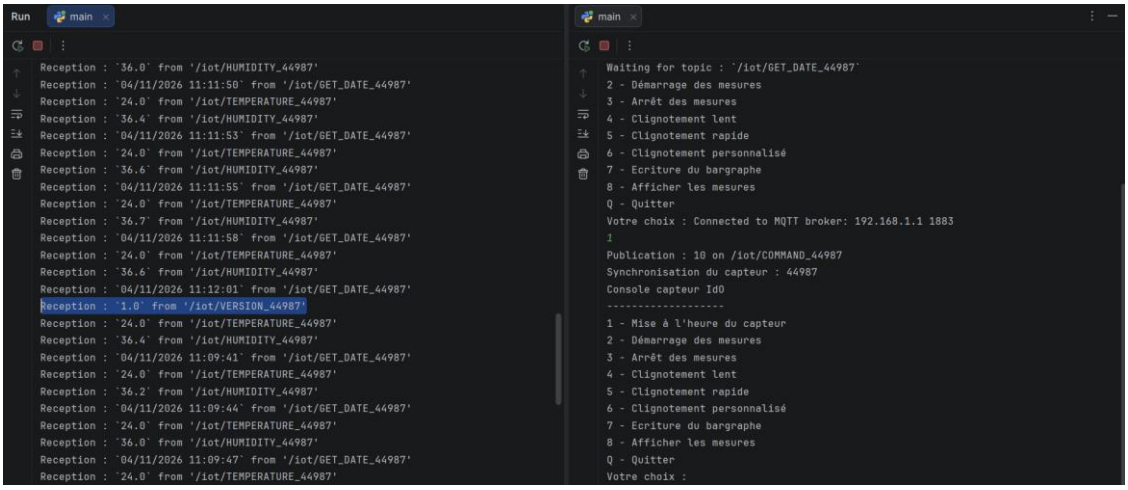


Figure 15 – *Lecture (à gauche) et écriture (à droite) en parallèle : la commande 10 fait publier la version et l'heure recule de 11:12:01 à 11:09:41*

Sur cette capture, juste après la commande 10, la version arrive puis la date publiée recule de 2 min 20 s. Le RTC du capteur avançait donc de plus de deux minutes, et la synchronisation NTP l'a remis à l'heure. Pour quelques

heures de fonctionnement, c'est une dérive importante. Les autres commandes ont été vérifiées de la même façon : 55 s sans publication entre l'arrêt (30) et le redémarrage (20), et les 10 LED allumées pour 1023.

4.3 Réception dans un thread

Pour n'avoir plus qu'un seul programme, la réception tourne maintenant dans un `ThreadMQTT`. Le `on_message` n'affiche plus rien : il range les valeurs dans les attributs `version`, `temperature`, `humidite` et `date_IdO` du client, et le thread principal vient les lire quand il en a besoin.

```
class ThreadMQTT(Thread):
    def run(self):
        self.mqtt.connect_sqlite()
        try:
            for topic in (topic_version, topic_temperature, topic_humidite, topic_datetime):
                self.mqtt.subscribe_mqtt(topic)
            while not self.stop:
                self.mqtt.wait_mqtt(timeout=1.0)   # rend la main chaque seconde pour tester stop
        finally:
            self.mqtt.disconnect_sqlite()
```

Il ne faut qu'une boucle réseau par client. Le `loop_start()` que nous avions ajouté au client d'écriture a donc été retiré, puisque le thread s'en charge. L'arrêt se fait en trois temps : `stopthread()` lève le drapeau, `join()` attend que le thread sorte de sa boucle, et la connexion est fermée ensuite. Au lancement, on voit d'ailleurs les messages du thread s'intercaler au milieu du menu : deux threads écrivent dans la même console.

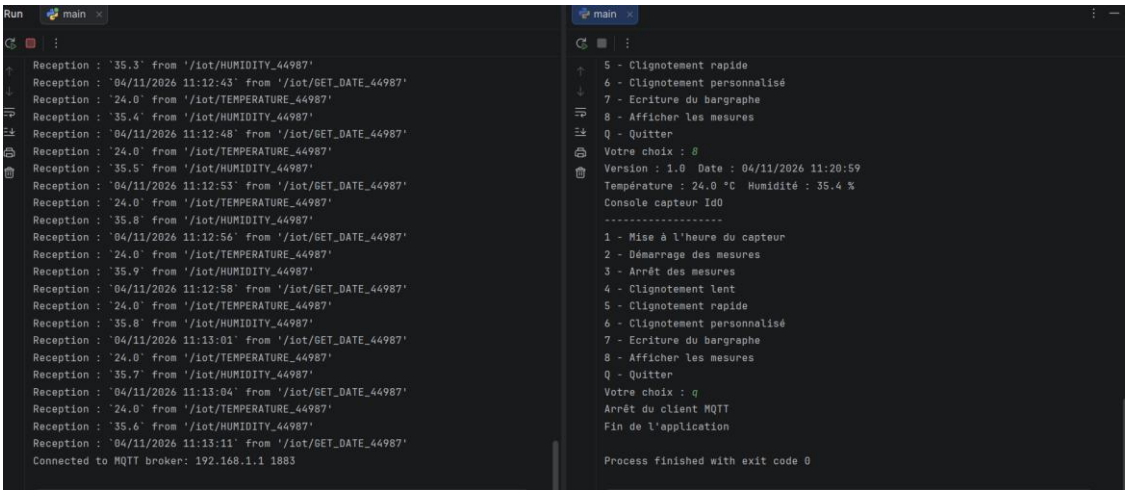


Figure 16 – *Choix 8 : valeurs lues par le thread ; choix Q : arrêt propre (exit code 0). À gauche, le client de lecture se reconnecte sans recevoir de données*

Cette capture nous a fait découvrir un défaut. À gauche, après une coupure, paho-mqtt reconnecte tout seul le client de lecture, mais plus aucune donnée n'arrive. La raison : nos abonnements sont faits une seule fois au démarrage, et non dans `on_connect`. Avec une session propre, le broker les oublie à la reconnexion.

4.4 Conception de l'interface

La fenêtre `mon_ihm.ui` a été dessinée dans Qt Designer (`pyside6-designer`), puis transformée en code par `pyside6-uic mon_ihm.ui -o mon_ihm.py`, qui produit la classe `Ui_MainWindow`. Elle est découpée en trois cadres (paramètres du broker, données du capteur, commandes), avec un menu et une barre d'état en bas. Nous avons gardé les noms de widgets du programme de l'énoncé (`btnConnect`, `btnSynchroDateTime`, `lneBargraph`…) pour pouvoir reprendre son squelette tel quel.

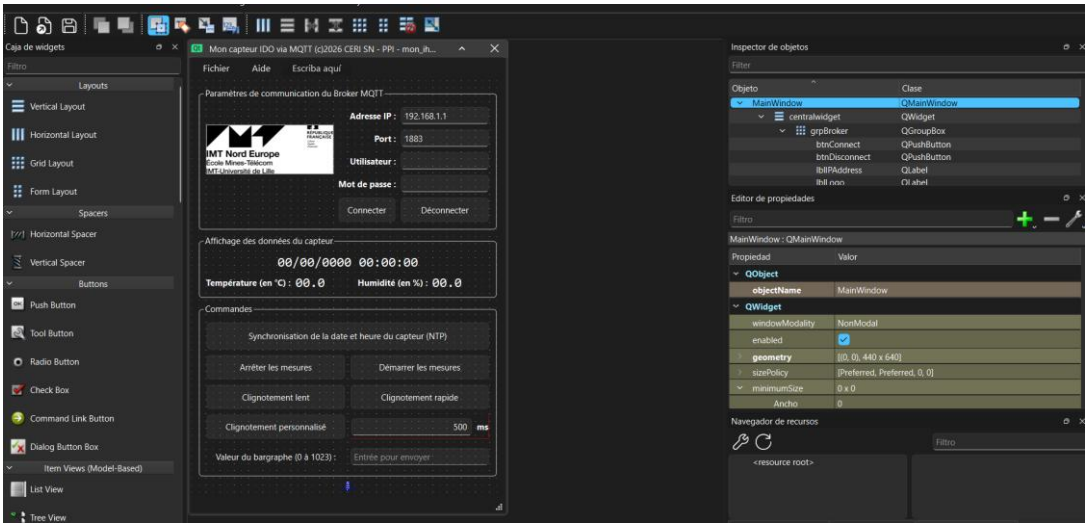


Figure 17 – *La fenêtre mon_ihm.ui dans Qt Designer, avec l'inspecteur d'objets et l'éditeur de propriétés*

Le logo est l'image `img/logo-noir.png`, rangée dans le projet et affichée dans un `QLabel` de taille fixe. Nous avons choisi une taille aux mêmes proportions que l'image pour qu'elle ne soit pas déformée par `scaledContents`. Au premier essai, l'image pointait vers notre dossier Téléchargements ; comme le chemin est relatif, le logo aurait disparu sur un autre PC.

4.5 Programme principal

`MainWindow` hérite à la fois de `QMainWindow` et de `Ui_MainWindow`. Dans le constructeur, on branche chaque signal sur une méthode : le `clicked` des boutons, le `returnPressed` du champ bargraphe, le `timeout` du `QTimer` et le `triggered` des menus.

Tableau 7 – *Méthodes de MainWindow*

Méthode	Fonctionnement
`connectMQTT`	Vérifie les champs, crée le client `Client_IHM44987` et se connecte. Si le broker ne répond pas, l'erreur s'affiche dans la barre d'état au lieu de faire planter le programme. Lance ensuite un nouveau `ThreadMQTT` et le timer de 1 s
`display`	Appelée par le timer : recopie dans la fenêtre la date, la température et l'humidité gardées par le thread
`disconnectMQTT`	Arrête le timer, puis `stopthread()` et `join()`, déconnecte, et remet tout à zéro pour pouvoir se reconnecter
`envoyer`	Publie si un client est connecté ; sinon affiche « Pas connecté » au lieu d'une erreur
Commandes	`synchroDateTime` (10), `startMeasure` (20, 50), `stopMeasure` (30, 40), `fastBlink` (50), `slowBlink` (40), `customBlink` (période puis 60), `setBargraph`
`closeEvent`	Appelée par Qt quelle que soit la façon de fermer la fenêtre : arrête le thread et la connexion. Sans elle, la fenêtre disparaissait mais le processus continuait de tourner

Un thread Python ne peut être démarré qu'une seule fois : réutiliser le même objet provoquerait une erreur à la reconnexion. Un `ThreadMQTT` neuf est donc créé à chaque connexion, et l'enchaînement Déconnecter puis Connecter a été testé sans problème. Nous avons aussi ajouté des validateurs (`QRegularExpressionValidator`) pour que l'utilisateur ne puisse pas taper n'importe quoi :

Tableau 8 – *Contrôle de saisie*

Champ	Expression régulière	Effet
Adresse IP	4 nombres de 0 à 255 séparés par des points	Pas d'adresse invalide

Champ	Expression régulière	Effet
Port	`\d{1,5}`	Que des chiffres, donc `int()` ne peut plus échouer
Période	`[1-9]\d{0,2}00`	Multiple de 100 ms, comme l'exige le capteur
Bargraphe	`\d{1,3}|10[01]\d|102[0-3]`	0 à 1023 ; la touche Entrée n'envoie rien si la valeur est fausse

4.6 Notice de fonctionnement de l'interface



Figure 18 – *Fenêtre « Mon capteur IDO via MQTT » connectée au capteur*

Tableau 9 – *Utilisation de la fenêtre*

Élément	Utilisation
Adresse IP, Port, Utilisateur, Mot de passe	Paramètres du broker, déjà remplis (192.168.1.1, 1883). Le mot de passe est masqué.
Connecter / Déconnecter	Connecter établit la liaison et lance la réception. La barre d'état indique « Connection au broker MQTT OK » ou l'erreur rencontrée. Après Déconnecter, l'affichage reste figé sur les dernières valeurs.
Affichage des données	Date et heure du capteur, température et humidité, mises à jour toutes les secondes.
Synchronisation NTP	Commande 10 : le capteur se remet à l'heure et publie sa version.
Arrêter / Démarrer les mesures	30 puis 40, ou 20 puis 50. La LED de vie clignote lentement quand la mesure est arrêtée et vite quand elle tourne.
Clignotement lent / rapide / personnalisé	Commandes 40 ou 50 ; pour le mode personnalisé, la période (multiple de 100 ms) part sur `BLINK_`, puis la commande 60.
Valeur du bargraphe	Taper une valeur de 0 à 1023 et appuyer sur Entrée : elle est publiée sur `BARGRAPH_`.
Fichier → Afficher les courbes (Ctrl+G)	Ouvre la boîte de dialogue des courbes (§4.8).
Fichier → Quitter (Ctrl+Q) ou ✕	Ferme la fenêtre en arrêtant proprement le thread et la connexion.

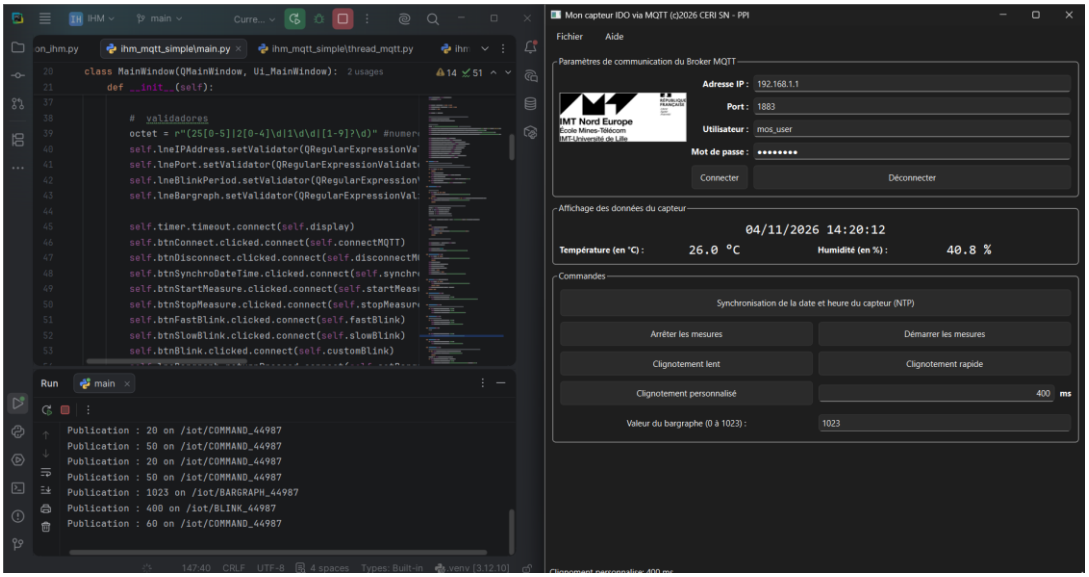


Figure 19 – *Commandes depuis la fenêtre : démarrage (20, 50), bargraphe 1023, clignotement personnalisé (400 puis 60)*



Figure 20 – *Bargraphe 1023 envoyé depuis l'IHM*

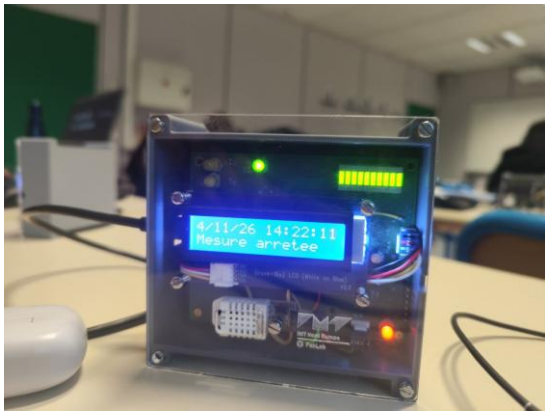


Figure 21 – *« Arrêter les mesures » depuis l'IHM*

4.7 Archivage dans SQLite

Chaque mesure reçue est enregistrée dans `IOT.db`, dans une table `IOT_DATA (Id_capteur, Date, Version, Temperature, Humidite)` créée au besoin. Trois décisions méritent une explication.

D'abord, la base est ouverte dans le thread. SQLite interdit d'utiliser une connexion dans un autre thread que celui qui l'a créée, et `on_message` s'exécute dans le `ThreadMQTT`. La connexion s'ouvre donc au début de `run()` et se ferme dans son bloc `finally`, même en cas d'erreur.

Ensuite, la ligne est écrite à la réception de `GET_DATE`. L'énoncé propose d'archiver au dernier topic publié, « généralement humidity », mais notre capteur envoie T, puis H, puis la date. En écrivant au moment de la date, T et H du même cycle sont déjà connues.

Enfin, la date est stockée au format ISO. SQLite compare les textes caractère par caractère : avec « 05/10/2026 » contre « 04/11/2026 », c'est le jour qui décide, pas le mois. Seul le format AAAA-MM-JJ HH:MM:SS se trie dans l'ordre du temps, ce dont les courbes ont besoin. La fenêtre, elle, continue d'afficher la date à la française.

```
def archiver(self, id_capteur):
    if self.conn is None or not self.temperature or not self.humidite:
        return
    date_iso = datetime.strptime(self.date_IdO, "%d/%m/%Y %H:%M:%S").strftime("%Y-%m-%d %H:%M:%S")
    self.conn.execute(f"INSERT INTO {self.tablename} VALUES (?,?,?,?,?)",
                      (id_capteur, date_iso, self.version, float(self.temperature),
float(self.humidite)))
    self.conn.commit()
```

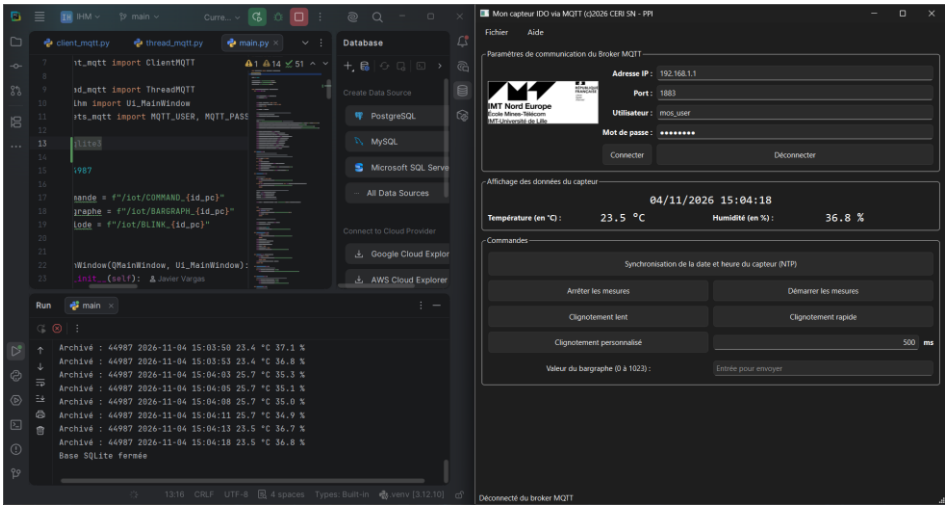


Figure 22 – *Archivage en direct (« Archivé : … ») puis fermeture de la base à la déconnexion*

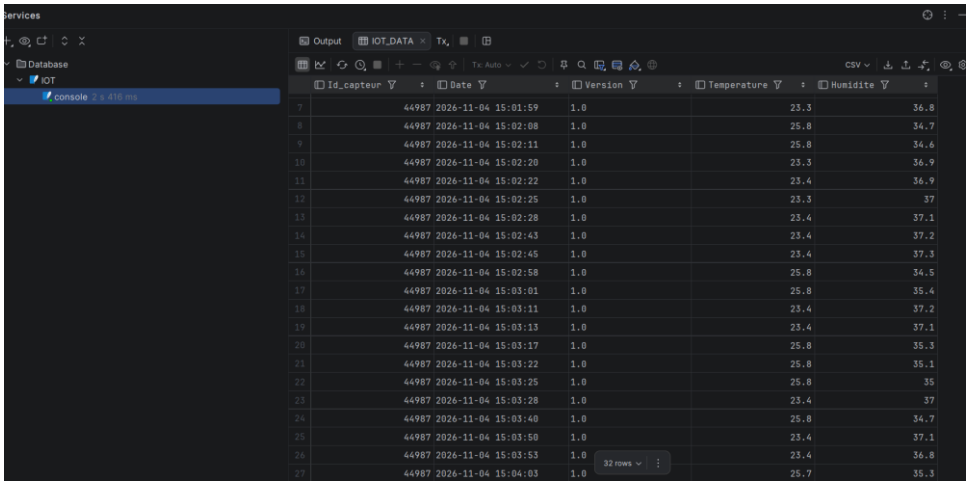


Figure 23 – *Table IOT_DATA ouverte dans PyCharm : 32 mesures entre 15:01:16 et 15:04:18*

4.8 Affichage des courbes

La boîte de dialogue `graphe.ui`, faite à partir du modèle « Dialog with Buttons Bottom », contient deux champs de date, une zone `QGraphicsView` et les boutons « Effacer les courbes » et « Charger les données ». On l'ouvre depuis le menu Fichier. Elle est modale, mais le thread continue d'archiver pendant qu'elle est ouverte. Le bouton Charger exécute :

```
SELECT Date, Temperature, Humidite FROM IOT_DATA
WHERE Id_capteur = ? AND Date BETWEEN ? AND ? ORDER BY Date ASC
```

Les champs de date affichent le format `yyyy-MM-dd HH:mm:ss`, le même que dans la base, sinon la comparaison ne renvoie rien. Les dates lues sont converties en `datetime` pour que l'axe des abscisses soit un vrai axe de temps. La figure Matplotlib est dessinée en mémoire (moteur `Agg`), convertie en image et posée dans la scène. Nous avons corrigé au passage quelques points du code de l'énoncé : la scène est créée dès l'ouverture (sinon, cliquer sur Effacer avant Charger ferait planter le programme), l'image est copiée avant que la figure ne soit libérée, et un message prévient quand l'intervalle ne contient aucune mesure.

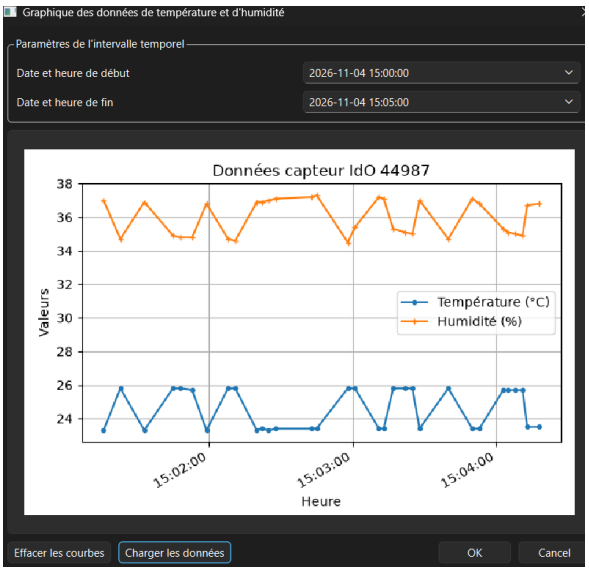


Figure 24 – *Courbes de température et d'humidité entre 15:00 et 15:05*

Ces courbes nous ont réservé une surprise. La température saute sans arrêt entre deux paliers, environ 23,4 °C et 25,8 °C, et au même instant l'humidité saute dans l'autre sens, entre 37 % et 34,8 %. Un seul DHT22 ne fait pas ça. Il y avait en fait deux émetteurs qui publiaient sur nos topics. Le broker ne vérifie pas qui publie : avec une ACL identique pour tous, n'importe quel client connecté peut écrire dans les topics d'un autre poste.

5 Difficultés rencontrées

Côté réseau, le serveur NTP de l'énoncé ne répondait pas depuis le réseau IoT, et il a fallu trouver le serveur local. Le changement d'heure du 25 octobre nous a aussi obligés à passer le décalage de 7200 à 3600 s. Nous avons perdu du temps avant de comprendre que le PC devait être lui aussi sur le réseau IoT pour voir le broker.

Sur l'Arduino, les erreurs étaient surtout des fautes d'inattention : le port MQTT écrit entre guillemets comme une chaîne, `connect()` confondu avec `connected()`, le texte de l'humidité écrasé par la date avant l'envoi, ou encore l'onglet des secrets enregistré en `.h.ino` par l'IDE.

En Python, il a fallu s'adapter à l'API de paho-mqtt 2.x, à la casse des topics, au thread qu'on ne peut lancer qu'une fois et à la connexion SQLite liée à son thread. L'environnement virtuel avait d'abord été créé en Python 3.15 au lieu de 3.12, et il a fallu exclure du dépôt Git les fichiers de configuration de l'IDE et les identifiants. Pour les courbes enfin, une virgule en trop dans la requête faisait disparaître la clause `WHERE` (« Incorrect number of bindings supplied »), et l'axe affichait des heures en double avant que nous passions à une graduation par minute.

6 Conclusion et perspectives

Au final, le capteur conçu en simulation puis porté sur la carte réelle mesure, affiche et publie ses données, et il obéit aux commandes reçues par MQTT. L'IHM permet de le suivre, de le piloter, d'archiver ses mesures et de les revoir sous forme de courbes. Le projet nous a fait toucher à beaucoup de choses en peu de temps : microcontrôleur et capteurs, simulation, WiFi et protocoles réseau, analyse de trames, programmation multithread, interface Qt et base de données.

Ce que nous retenons surtout, c'est l'intérêt du modèle publication/abonnement : le capteur ne sait rien de ceux qui l'écoutent, ce qui a permis d'ajouter l'IHM sans toucher au programme Arduino. Nous avons aussi appris à structurer un programme embarqué sans `delay()` bloquant, et à séparer l'interface de la communication avec un thread. Avec le recul, la sécurité nous paraît être le point faible de l'installation, et l'émetteur parasite des courbes l'a bien montré.

Si nous devions continuer, nous commencerions par la sécurité : MQTT sur TLS (port 8883), un identifiant par objet et des ACL par client (`%c`, `%u`), pour qu'un poste ne puisse plus publier chez le voisin. Ensuite, nous déplacerions les abonnements dans `on_connect` pour qu'ils reviennent après une reconnexion, et nous resynchroniserions le RTC

régulièrement. Enfin, il resterait le challenge : envoyer date, température et humidité dans un seul message sur `DATA_`, s'abonner à `/iot/#` et pouvoir choisir dans une liste le capteur de la salle à afficher.